

Lab: Ensemble Methods

Rishi Kalra

Note: In all the three tasks, the submissions will be relatively graded based on the MAE achieved on the hidden test-cases which aren't available to you

Task 1 Gradient Boosting using Huber Loss

You will implement a gradient boosting ensemble from scratch that handles both **regression** (Huber loss). The ensemble takes the form

$$f(x; \theta) = \sum_{m=1}^M \beta_m F_m(x; \theta_m) \quad (1)$$

where $\beta_m = \text{learning_rate}(m)$ and each F_m is a weak learner fitted on the pseudo-residuals of the previous stage.

Rules.

- Edit `student.py` only. Do not rename any function, class, or variable.
- Run `python autograder.py` to check your score before submitting.
- You may *not* use `GradientBoostingRegressor` or any pre-built boosting library.

Part A Loss Functions

A1 `huber_loss(residuals, delta)`

The Huber loss for a single residual r is:

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta|r| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \quad (2)$$

Implement `huber_loss` to return the **mean** L_δ over all elements of `residuals`. You can tune this `delta` parameter.

Verification: `huber_loss([-0.5, 2.0, -3.5], delta=1.0)  $\approx$  2.0417.`

A2 `huber_pseudo_residuals(y_true, y_pred, delta)`

The negative gradient of L_δ with respect to $\hat{y}_i$ is:

$$g_i = \begin{cases} r_i & |r_i| \leq \delta \\ \delta \cdot \text{sign}(r_i) & \text{otherwise} \end{cases} \quad r_i = y_i - \hat{y}_i \quad (3)$$

Return an array of the same shape as `y_true`.

Part B GradientBoostingEnsemble Class

B1 `learning_rate(m)`

Return the coefficient β_m for stage m (1-indexed). You **design this function yourself** — it can be a constant, a decaying schedule, or any other positive function of m .

Autograder checks: $\beta_m > 0$ for all $m \in [1, 50]$.

B2 `_compute_pseudo_residuals(y_true, f_pred)`

Compute the negative gradient of the Huber loss w.r.t. `y_pred`.

B3 `fit(weak_learners, X, y)`

Implement the boosting loop:

1. Initialise the running score f , $f_i = \bar{y}$ for all i ; store in `self.initial_prediction_`
2. Scale the training dataset using `StandardScaler()`.
3. For $m = 1, \dots, M$:
 - a. Compute pseudo-residuals $g = \text{_compute_pseudo_residuals}(y, f)$
 - b. `clone` the m -th weak learner and fit it on (X, g)
 - c. Update $f \leftarrow f + \beta_m \cdot F_m(X)$
 - d. Append the fitted clone to `self.models_`
4. Return `self`

B4 `predict(X)`

Return raw score $\hat{y} = \text{initial_prediction_} + \sum_m \beta_m F_m(x)$.

B5 `staged_loss(X, y)`

Replay the boosting loop on (X, y) , recording the loss after each stage. Return a `np.ndarray` of shape $(M,)$ containing the huber loss of residuals at stage m

Part C Design Your Weak Learner Lists

Define module-level list in `student.py`:

```
1 REGRESSION_LEARNERS: List[BaseEstimator] = [...]
```

Requirements:

- At least 5 estimators
- Any sklearn-compatible *regressor* (pseudo-residuals are always real-valued)
- No pre-built boosting models (`GradientBoostingRegressor`, `XGBoost`, etc.)

Candidate base learners: `DecisionTreeRegressor`

The autograder runs your full pipeline end-to-end and reports MAE and accuracy — these are informational and do not affect your score, but they are a good sanity check.

Task 2 Dynamic Ensemble Selection

Overview

In this assignment, you will implement a **Dynamic Ensemble Selection (DES)** method for **regression**, specifically the **K-Nearest Oracles Union (KNORA-U)** algorithm.

Unlike static ensembles (e.g. averaging), KNORA-U dynamically selects a subset of models *for each test sample* based on their *local competence* in the feature space.

You will:

- Build a pool of diverse regression models
- Optionally apply bootstrap sampling to distinguish identical model architectures
- Implement the KNORA-U selection mechanism
- Evaluate performance using Mean Absolute Error (MAE)

Dataset

You are provided with a regression dataset-

- **Training data:** `train.npz`
- **Test data:** `test.npz`

Each file contains:

$$X \in \mathbb{R}^{n \times d}, \quad y \in \mathbb{R}^n$$

Important Rules

- You **must not** use test data during training or model selection
- Dynamic selection must rely only on scaled training data (DSEL)

Base Learners

You may use only the following regressors:

- Linear Regression
- Ridge Regression
- Lasso Regression
- Decision Tree Regressor
- k-Nearest Neighbors Regressor
- Support Vector Regressor (RBF kernel)

Hyperparameters must lie within the ranges specified in the starter code.

Bootstrapping Requirement

If the same model architecture appears multiple times in your ensemble (e.g. two decision trees with identical hyperparameters), then:

- **Bootstrap sampling is mandatory**
- Each instance must be trained on a different bootstrap sample

This improves diversity and prevents degenerate ensembles.

Dynamic Ensemble Selection

Concept

Dynamic Ensemble Selection chooses models *per test sample*, instead of using the same ensemble for all inputs.

The key idea:

“A model that performs well globally may not be optimal locally.”

KNORA-U for Regression

Definitions

Let:

- $\mathcal{M} = \{m_1, \dots, m_L\}$ be the pool of trained regressors
- x_q be a query (test) sample
- $\mathcal{D}_{\text{DSEL}}$ be the dynamic selection dataset (training data after scaling)
- K be the neighborhood size
- ϵ be the local error tolerance

Algorithm

For each test sample x_q :

1. Find the K nearest neighbors of x_q in $\mathcal{D}_{\text{DSEL}}$
2. For each model m_i , compute the local MAE:

$$\text{MAE}_i = \frac{1}{K} \sum_{j=1}^K |m_i(x_j) - y_j|$$

3. Select all models satisfying:

$$\text{MAE}_i \leq \epsilon$$

4. If no model satisfies the criterion, select the single best model:

$$\arg \min_i \text{MAE}_i$$

5. Predict by averaging the selected models:

$$\hat{y}(x_q) = \frac{1}{|\mathcal{S}|} \sum_{m \in \mathcal{S}} m(x_q)$$

This is called the **Union** rule because *all competent models* are used.

What You Must Implement

You are required to implement the following components in `student.py`:

Distance and Neighbors

- Euclidean distance computation
- k-Nearest Neighbor index selection

Bootstrapping Utilities

- Bootstrap sampling
- Conditional training with or without Bootstrapping

KNORA-U Regressor

- Model storage and DSEL setup
- Local competence estimation
- Dynamic model selection
- Prediction via averaging

Model Pool Construction

- At least 5 fitted regressors
- Correct use of Bootstrapping for duplicate architectures

Evaluation

Your submission will be evaluated based on:

- **Correctness of KNORA-U implementation**
- **Model diversity** (non-trivial prediction differences)
- **Final MAE on test data**

Degenerate solutions (e.g. identical models, static predictions, or adversarial logic) will be penalized.

Task 3 ”Just Beat It!”

Task Objective

The objective of this task is deliberately simple:

Beat the baselines established in the previous two methods.

You have already implemented:

- A **global ensemble** (boosting-style model)
- A **dynamic ensemble selection** method (KNORA-U)

Your job now is to design an ensemble fusion strategy that achieves a **lower Mean Absolute Error (MAE)** than *both* of these baselines.

What You Are Given

You are provided with:

- A template file `student.py`
- An autograder that evaluates predictions on a hidden test set
- A fixed set of allowed regression models

The autograder will call a single function:

```
task3_predict(X_train, y_train, X_test)
```

and compute the MAE of your predictions.

What You Must Implement

1. Model Pool

You must define a list:

```
FUSION_LEARNERS
```

This list must satisfy:

- At least **5** unfitted models
- All models must belong to the allowed model set
- No pre-built ensemble methods (e.g. `GradientBoostingRegressor`)

You are free to choose model types and hyperparameters.

2. Fusion Strategy

You must implement a fusion mechanism inside:

```
task3_predict(...)
```

Your fusion strategy may include (but is not limited to):

- Simple averaging
- Weighted averaging
- Stacking / meta-learning
- Local or dynamic weighting
- Hybrid methods inspired by KNORA-U

As long as it is valid and improves MAE, it is allowed.

Rules and Restrictions

- **Test labels must never be used**
- You may not modify the autograder
- You may not rename required functions or variables
- You may not hard-code predictions or exploit dataset specifics

Violations will result in zero credit.

Evaluation Criteria

Your submission will be evaluated on:

1. **Final MAE on the test set**
2. Correctness of implementation
3. Robustness and non-degeneracy

To pass, your method must **outperform both prior baselines**.

Important Insight

If your ensemble diversity score is zero, it means:

- Your models are too similar, or
- Your fusion reduces to trivial averaging

Diversity and intelligent fusion are essential.

Final Note

This task is intentionally open-ended.

There is no single correct solution — only better ones.

JUST BEAT IT!

— *Michael Jackson*